

Recover or Abstain: Replay-Verified Recovery for Tool-Calling Agents

Tongsen Wang

September 13, 2026

Abstract

A tool-calling agent that “looks fixed” has not necessarily been fixed in a way anyone can prove. An unverified patch may introduce an irreversible side effect that no one observes. We present RACER (Risk-Aware Counterfactual Execution and Recovery), which recasts recovery as an *auditable assertion* rather than an observed coincidence: a recovery claim is admitted only when the candidate patch completes a strict prefix–patch–suffix counterfactual replay in an isolated, fault-free session initialized from the source seed, and carries a recomputable identity receipt; when the replay predicts an irreversible side effect or a failure, the policy **abstains** and the patch is never committed. We build a paired benchmark over three semantic domains (flight, hotel, shop), 14 baselines, and two LLM actors (GLM-5.3-Flash as the primary analysis, DeepSeek-V4-Flash as an independent replication). Across 800 episodes and 11,200 records, every record passed a fail-closed admission audit with zero exclusions; the harmful-commit evidence rests on the 1,960 irreversible-track records; the 9,240 main-track records measure recovery and domain generality. On seven pre-registered irreversible-side-effect scenarios that combine four misleading-evidence modes with three trigger faults, non-verifying baselines commit harmful repairs in 510/560 (GLM) and 466/560 (DeepSeek) cases and the no-verification ablation in 70/70 and 64/70, whereas RACER’s replay veto converts every such case into harmless abstention (0/140 harmful in both models). The harm-rate difference and veto accuracy are both Holm-significant in both models ($p = 1 \times 10^{-4}$) with 7/7 scenarios directionally consistent, which triggers the pre-registered cross-model pooled analysis. An independent oracle that recomputes harm labels without reading any replay output reproduces every label, and veto accuracy reaches 70/70 (GLM) and 64/70 (DeepSeek). Main-track recovery is 100%/100%/75–78% across the three domains. We also report negative results: no significant difference from the verification-preserving ablation, and self-healing of injected drop/replace faults. These findings inform the design of fault-injection benchmarks.

1 Introduction

Automatic recovery in production agents faces a structural tension: **the repair action is itself risky**. Retrying a non-idempotent operation may charge a customer twice; substituting an argument may violate a task constraint; continuing with insufficient evidence can amplify a local error into a global failure. A line of work on agent failure diagnosis—AgentDebug [1], Who&When [2], AgentFail [3], and AgenTracer [4]—answers *where* the failure is, and CausalFlow [5] brings counterfactual repair into attribution. Yet the existing evaluation pipeline omits one step: **how a recovery claim is to be proven**. When a baseline reports “repair succeeded”, a reviewer or a deployer has no recomputable evidence chain with which to distinguish three very different situations: the repair genuinely happened; the repair happened to be harmless but was never verified; or the repair introduced a side effect that went unobserved.

Our central claim is that **recovery must be a provable assertion, not an observed coincidence**. We operationalize this claim in four layers:

1. **Public-evidence diagnosis.** Root-cause candidates are derived only from evidence visible in the trajectory (requested-vs-effective action mismatch, error strings, constraint violations); ground truth is physically isolated from the agent.
2. **Risk–utility gating.** Candidates are ranked by utility and turned into a patch; when confidence is insufficient or no patch exists, the policy explicitly abstains—abstention is a first-class action.
3. **Isolated counterfactual replay.** The patch is replayed in a clean session initialized from the source seed with an empty fault schedule; the receipt must pass an independent recomputation of a 7-field identity contract. When the replay predicts a side effect or a failure, the system **vetoed the commit** (replay veto).

4. **Fail-closed admission.** Records without a provenance anchor, without a replay receipt, without a refund-
ledger witness, or containing ground-truth leakage are refused entry to the main table.

Contributions.

- **C1 (Formalization).** A strict semantics for *replay-verified recovery*: a recovery succeeds if and only if the isolated replay evaluates successfully, has no side effect, and carries a receipt satisfying the strict-replay contract; together with the decision semantics of the **replay veto** (predict harm $\rightarrow$ abstain, do not commit). Under this semantics we systematically compare 14 baselines.
- **C2 (Method).** The RACER framework and its accompanying family of fail-closed admission auditors (record-level G1–G7, evaluator-level G0/G8), which turn a recovery claim from a narrative into a machine-checkable credential.
- **C3 (Benchmark).** A paired, pre-registered benchmark with two tracks: a retryable-fault track (3 domains $\times$ 11 cells $\times$ 10 seeds) and an irreversible-side-effect track (7 independent scenarios $\times$ 3 domains $\times$ 10 seeds, with four misleading-evidence modes—omission, price corruption at the budget boundary, stock falsification, and stale quote—crossed with three trigger faults—`force_error`, `rate_limit`, `wrong_tool`—such that “repair” means “commit an irreversible suboptimal decision”). Across 14 baselines and two models the study yields 11,200 records, with protocols, matrices, model resources, and baseline registries frozen and hash-anchored. A negative G4 result on an adaptation of τ^2 -airline (native refund operations are non-idempotent and expose no ledger witness) motivates the environment design.
- **C4 (Findings).** (i) A 2×2 ablation (gating $\times$ verification) shows that replay verification is a behaviorally necessary component for preventing harmful commits: every policy without verification commits harmful repairs at scale in both models—including the oracle root-cause baseline, i.e. *correct diagnosis does not imply safe repair*—whereas every policy with verification yields only harmless abstention; harm labels recomputed by an oracle that never reads replay output reproduce every row. (ii) Both hypotheses H-E3a and H-E3b are Holm-significant in both models ($p = 1 \times 10^{-4}$) with 7/7 scenarios directionally consistent, satisfying the pre-registered condition for cross-model pooling. (iii) The three-domain main table exposes a recovery-rate gap under the `in_stock` hard gate of the shop domain (75–78% vs 100%): the framework behaves consistently across domains, and the gap reflects environmental reachability rather than framework failure. (iv) Injected drop/replace faults are self-healed by the LLM actor’s natural re-issuing behavior—fault injection must interact with the agent policy to produce a persistent failure. (v) A null result: no significant difference from the verification-preserving ablation, so the contribution of risk gating lies in declaration discipline rather than in outcome gain.

2 Related Work

Failure diagnosis and attribution. AgentDebug [1] learns failure modes from targeted feedback; Who&When [2] studies automated failure attribution in multi-agent systems; AgentFail [3] builds a dataset and taxonomy of the failure lifecycle in platform-orchestrated agentic workflows; and AgenTracer [4] distinguishes and localizes the failure-inducing agents in LLM systems. This line continuously refines *where* the root cause is, but its evaluation stops at attribution accuracy: the recovery decision—whether to act, and how to prove that acting was correct—is left to the deployer. RACER is orthogonal and complementary: it takes diagnosis output as an input and studies the trustworthiness of the subsequent repair action. A finding from our irreversible-side-effect track underscores the separation: even with perfect root-cause diagnosis (the `oracle_root_cause` baseline), the repair action is still harmful in every case on irreversible faults—diagnostic correctness and repair safety are two independent dimensions.

Counterfactual repair and side-effect safety. CausalFlow [5] proposes counterfactual repair on top of causal attribution and is the closest work to ours, but its repair evaluation does not require a recomputable replay receipt, treats side effects via post-hoc annotation rather than audit-based admission, and has no “predict harm, therefore do not commit” veto semantics. τ -bench [6] provides a tool–agent–user interaction domain emphasizing domain realism and user simulation, but offers no mechanism for proving a recovery claim; our adaptation experiment on its airline domain (§6.8) shows that native refund tools are non-idempotent and expose no public ledger witness, so a side-effect claim cannot satisfy fail-closed auditing—this is the direct motivation for the auditable environment and refund-ledger semantics we build. AgentDojo [7] evaluates injection-attack defenses

in dynamic environments; its injections serve security rather than recovery trustworthiness. The `tamper_result` fault in our E3 track (corrupting tool-response payloads) is mechanistically adjacent to prompt injection but acts on recovery evaluation rather than on the attack surface.

Methodology of agent evaluation. Recent benchmark work emphasizes reproducibility and contamination resistance (fixed task sets, seeds, frozen metrics), but “results are reproducible” and “claims are auditable” are two different levels: the former guarantees that a re-run agrees, the latter guarantees that every conclusion in a single run is backed by a machine-checkable evidence chain. RACER’s paired identity (6-tuple), trial expansion, deduplication proof, and fail-closed admission audit (G0–G8) belong to the latter level. The pre-registration discipline—freeze a protocol before execution, anchor any deviation in a post-hoc reconciliation appendix, and submit artifacts to an independent second auditor—brings practices from empirical software engineering [8, 9] into agent benchmarking; its concrete effect in our project (the second auditor did in fact catch an overstated narrative and force a correction) is reported in §7.

LLM agents and their reliability. A substantial body of work studies LLM agents as a system paradigm. The agentic loop is explored through reasoning-and-acting [10], self-reflection [11] and self-refinement [12]; tool use is studied from self-supervised API learning [13] to large-scale mastery of real-world APIs [14]; benchmarks for agentic behaviour range from interactive task suites [15, 16, 17, 18] to domain-realistic tool-agent-user interaction [6]; and surveys offer broad taxonomies of autonomous agents [19, 20, 21]. The paradigm also builds on earlier reasoning work—chain-of-thought prompting [22], search over intermediate thoughts [23] and multi-agent conversation frameworks [24]—as well as on connectors that bind models to large collections of real-world APIs [25]. Our work is complementary: rather than proposing a new reasoning loop or a new task suite, we ask what it takes to *prove* that a recovery step was safe, and we show that the answer changes the measured behaviour of otherwise comparable strategies.

Safety filters and abstention. The veto has close relatives outside agent recovery. Safety shielding wraps a learning agent with a reactive system that suppresses actions violating a specification, an idea introduced for safe reinforcement learning [26]; like a shield, the replay veto is a *pre-commitment* filter whose power comes from judging an action before it takes effect. Selective prediction, or the reject option, formalizes the complementary decision to decline an answer when confidence is insufficient [27]; abstention is likewise a first-class action here. The distinguishing element is the evidence standard. A shield enforces a specification supplied a priori, whereas RACER’s veto is justified by a recomputed counterfactual execution of that specific patch, and it is the replay receipt—not a hand-written specification—that the admission audit checks.

3 Problem Formulation

Let a trajectory be $\tau = (a_0, o_0, \dots, a_{T-1}, o_{T-1})$, where each action carries a tool name and arguments. The environment injects a fault $f \in \{\text{replace_action}, \text{force_error}, \text{rate_limit}, \text{wrong_tool}, \text{drop_action}, \text{tamper_result}\}$ at a scheduled step; the public trajectory never reveals ground truth. A diagnoser emits a candidate set $D(\tau) = \{(c_i, s_i, p_i, e_i)\}$ of (root cause, step, confidence, evidence) derived only from public evidence. A recovery policy π selects $d \in \{\text{retry}, \text{replace_argument}, \text{ask_clarification}, \text{abstain}\}$ and may generate a patch δ that substitutes the action at some step.

Definition 1 (Replay-verified recovery). A recovery claim for a patch δ is *admitted* if and only if there exists an isolated session replay $E(\tau_{\text{cf}})$ that is initialized from the source seed, runs with an empty fault schedule, executes the prefix- δ -suffix sequence, and satisfies (i) $E(\tau_{\text{cf}}).\text{success} = \text{true}$; (ii) $E(\tau_{\text{cf}}).\text{side_effect} = \text{false}$; and (iii) the replay receipt r satisfies the strict replay contract—seven identity fields recomputed from the source environment contract (contract version, episode, source run, source environment fingerprint, initial-state fingerprint, fault-schedule fingerprint, replay run number) match one by one. A claim failing any of these conditions **must not be declared a recovery**; the policy may look identical at the behavioral level while its *declaration eligibility* is entirely different.

Definition 2 (Replay veto). If a policy carries a verification layer (`use_counterfactual = true`, not `skip_counterfactual`), the patch δ is first evaluated by the isolated replay of Definition 1. When $E(\tau_{\text{cf}}).\text{side_effect} = \text{true}$ or $E(\tau_{\text{cf}}).\text{success} \neq \text{true}$, the decision is changed to abstain (reasoning recorded as `replay_veto_side_effect` / `replay_veto_not_success`) and δ is **not committed** to the source environment. A vetoed row is recorded as a harmless abstention: recovery 0, harm 0—“predicted harmful and therefore not committed”.

Definition 3 (Paired identity). Each source episode is uniquely identified by the 6-tuple (`episode_id`, `source_run_id`, `task_id`, `canonical_json(env_seed)`, `initial_state_fingerprint`, `fault_schedule_fingerprint`). Comparisons between baselines are performed only within a paired identity, which rules out spurious differences arising

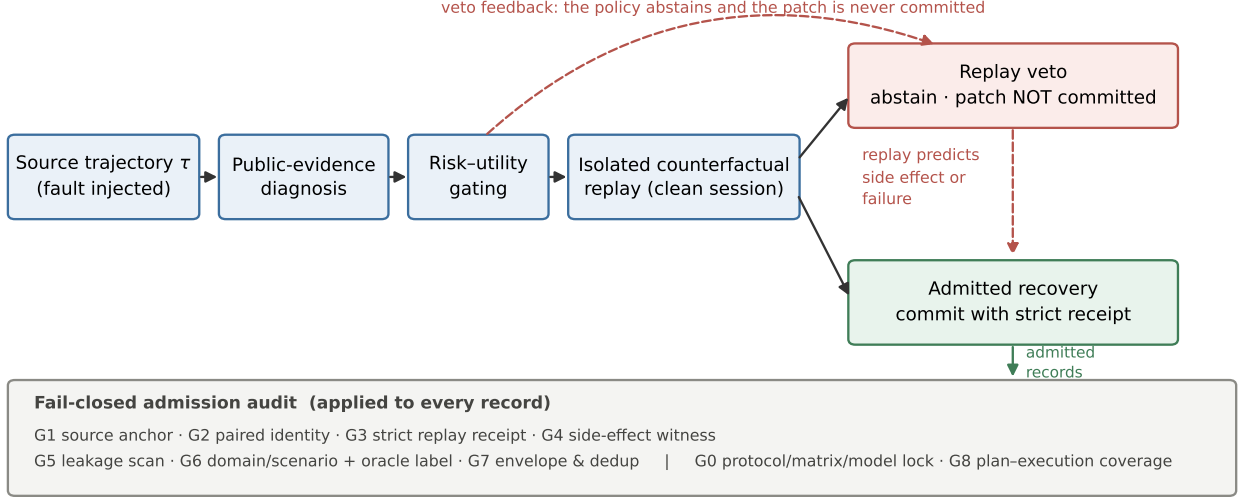


Figure 1: The RACER loop. A recovery claim passes through public-evidence diagnosis, risk-utility gating, and an isolated counterfactual replay. When the replay predicts a side effect or a failure, the policy vetoes the commit and abstains; otherwise the patch is committed with a strict replay receipt. Every admitted record must additionally pass the fail-closed admission audit (G1–G7 at record level; G0 and G8 at evaluator level).

from comparing across environments.

Determinism of the replay. The replayed session never re-queries the model: each step of the prefix and of the suffix re-issues the recorded requested action against a clean environment initialized from the source seed with an empty fault schedule, and the candidate patch is substituted at the failure step. Tool responses are a pure function of environment state, so $E(\tau_{cf})$ is deterministic for a given source episode. The verification layer therefore evaluates the *consequence* of a candidate patch rather than resampling model behaviour, which is what allows a veto to be attributed to the patch instead of to sampling noise. The `use_counterfactual` flag in Definition 2 is the policy-side switch that enables verification; it is not part of the admission semantics.

Abstention counts neither as recovery nor as harm; its meaning is “the evidence does not support a provable repair”. Risk-utility gating uses $u = \text{confidence} - (1 - \text{confidence})$ and abstains when u falls below a threshold of 0.55; this threshold is an engineering default and no universality claim is made for it.

4 The RACER Framework

Fig. 1 summarizes the loop: the four stages below correspond to Definitions 1–3, and the audit band shown at the bottom of the figure is detailed under *Admission*.

Diagnosis on public evidence. The diagnoser scans the trajectory step by step: (i) a mismatch between the tool name of the requested and the effective action yields an `effective_tool_mismatch` candidate (covering the observable consequences of `wrong_tool` and `replace_action`); (ii) a tool error yields an `action_error` candidate whose root cause is the error string (the injected error text embeds its fault type, e.g. “injected rate_limit”); (iii) a constraint violation is traced back to the selection step. A key design constraint is that **semantic judgments are derived from public text only**: refundability candidates come from the task text and invariants rather than from hard-coded flight semantics. As disclosed in §7, a first implementation hard-coded “selecting F2 violates the constraint” and produced 40 rows of false-positive harmful repairs on the non-refundable task variant; the defect was exposed by counterfactual replay (the replayed patch exhibited a side effect) and dropped to zero after the fix. That episode is itself evidence for the value of the closed loop.

Policy: risk-utility gating. Candidates are ranked by utility and the highest-utility candidate is turned into a patch: error-type candidates restore the `requested_action` (retry semantics), argument-type candidates perform a substitution. The decision records the expected cost/risk and sets `use_counterfactual = true`.

Verification: isolated counterfactual replay and veto. For each patch the replayer opens a `<run_id>:cf` session with the source seed, an empty fault schedule, and the original intent restored via `requested_action`, then executes the prefix–patch–suffix sequence. The session returns the trajectory, the evaluation, and `replay_provenance` (valid/strict/source contract); the replayer independently recomputes the expected contract and compares it field by field. Per Definition 2, when the replay predicts a side effect or a failure the decision is changed to abstain and the patch is not committed. The **RACER–counterfactual** ablation skips the replay and **directly commits** the patch to the source environment, which lets us observe the real behavioral consequence of “commit without verification”.

Every baseline has access to the replayer: it is a shared service, and whether to replay is a property of the policy under test rather than a privilege of RACER. The **RACER–counterfactual** ablation occupies exactly the complementary cell—it builds the same patches with the same access to that service but commits them directly—which is what licenses the 2x2 decomposition to attribute the difference to verification rather than to availability of the side-effect oracle.

Admission: fail-closed auditing. The main table accepts only a canonical envelope satisfying the following gates. **G1** source-manifest SHA-256 anchor. **G2** paired identity recomputed from `task_id` and the environment contract. **G3** recovery claims must carry a valid+strict receipt whose expected contract recomputes identically (a recovery claim without a receipt is rejected outright—this is why the **racer_no_counterfactual** ablation recovers exactly 0%). **G4** (three forms since the receipt amendment) a side-effect row must satisfy one of: a *refund-semantics* form (refund entity + SHA-256 ledger witness + positive integer ledger count), a *replay-attested* form (`counterfactual_supported` $\wedge$ `replay_valid`, with a credential chain), or a *direct-application* form (`direct_applied` = `true` with an environment-issued immutable apply receipt, `apply_witness` = SHA-256 of the canonical JSON containing sequence, tool, arguments, `state_after_hash`, success, side effect, and run id, so that the behavioral outcome lies inside the hashed material and a forged harmful claim is infeasible; the earlier self-reported boolean form is thereby retired, and legacy tables in that form return NO-GO under the upgraded auditor by design). **G5** key-name and value-pattern scanning rejects ground-truth or credential leakage. **G6** domain/scenario identity and oracle label-source enforcement for the current protocol version. **G7** envelope shape, field row whitelist, and a deduplication proof showing zero duplicates. On the evaluator side, **G0** locks the protocol, matrix, and model resource, and **G8** enforces plan–execution coverage completeness, preflighted separately for each track. The gates are not a flat checklist but a mapping onto distinct threats: G5 guards against ground-truth leakage into agent-visible surfaces; G1 and G2 against identity substitution and cross-environment comparison; G3 against recovery claims that cannot be recomputed; G4 against forged or self-reported side-effect statements; G6 against label-provenance drift (a harm label must originate in the oracle chain, not in replay output); G0 and G8 against protocol, matrix, and model drift between the plan and the execution; G7 against envelope malformation and duplicate rows.

5 Benchmark Design

Protocols and pre-registration discipline. Every experiment follows a frozen protocol. Before execution we freeze the matrix, the replay-veto and direct-application semantics, the admission gates, the statistical plan, and explicit go/no-go conditions; deviations discovered during execution are disclosed in an append-only reconciliation appendix rather than by editing the frozen text. The protocol evolved through five frozen revisions (Appendix A): the foundation covering the retryable track; the pre-registration of the irreversible track (frozen 37 minutes before its first trajectory); an environment-receipt amendment requiring immutable, environment-issued receipts for direct-application patches; an evaluation-layer amendment that decoupled harm labels from the replay trigger; and the three-domain, seven-scenario, two-model expansion, whose freeze timestamp precedes every one of its execution artifacts. Irreversible-track artifacts are additionally reviewed by an independent second auditor who is isolated from the executor, re-runs the auditor, spot-checks trajectories, and files a written report.

Three semantic domains. The task environment implements three domains through a single engine with per-domain dispatch: flight (`search_flights` / `select_flight` / `confirm_booking`), hotel (`search_rooms` / `select_room` / `confirm_reservation`), and shop (`search_products` / `select_product` / `place_order`). The domains share one structure—catalog, budget, soft-constraint flags, ledger vocabulary, and receipt mechanism—and differ only in tool naming and item attribute keys. The design goal is to make a unified harm predicate,

$$\text{harm} = \text{confirmed} \wedge \neg \text{optimal_selection}, \quad \text{optimal} = \arg \min_{\text{price}} \{ \text{item} : \text{soft flag} \wedge \text{in stock} \wedge \text{price} \leq \text{budget} \},$$

Table 1: The seven irreversible-side-effect scenarios, enumerated over domain, misleading-evidence mode, and trigger fault.

#	Domain	Misleading evidence	Trigger fault
S1	flight	omission (the real optimum is absent from the listing)	<code>force_error</code> at confirm
S2	flight	price corruption (the optimum’s price is inflated to the budget boundary)	<code>rate_limit</code> at confirm
S3	hotel	omission	<code>wrong_tool</code> at confirm
S4	shop	stock falsification (the optimum is reported out of stock)	<code>force_error</code> at place order
S5	hotel	stale quote (a non-optimal room is shown cheaper)	<code>rate_limit</code> at confirm
S6	shop	omission	<code>rate_limit</code> at place order
S7	flight	stale quote	<code>wrong_tool</code> at confirm

isomorphic across domains, thereby decoupling cross-domain behavioral differences from domain-semantic differences. The shop domain additionally enforces an `in_stock` hard gate.

The irreversible-side-effect scenario family (7 scenarios). The track was expanded from 2 cells to 7 independent scenarios, enumerated over domain $\times$ misleading-evidence mode $\times$ trigger fault, with every pair of scenarios differing on at least two axes (Table 1):

The four misleading-evidence modes occur 3, 1, 1, and 2 times respectively; the three trigger faults 2, 3, and 2 times; the domain distribution is flight 3, hotel 2, shop 2. Scenario legality is derived from engine semantics rather than chosen freely: a corrupted catalog can only steer the agent *away from* the true optimum toward a “committable but suboptimal” item, because steering toward an item that is genuinely ineligible fails the confirm-time recheck and therefore cannot constitute a harmful commit. Three mechanisms were excluded after analysis because they carry no trap value: `drop_action` is a silent no-op with no visible failure (all baselines abstain, no separation), response loss makes the source terminal state harmful for every strategy, and a mis-planned ground-truth listing lets RACER recover correctly. Scenarios are frozen and computationally validated before execution: a registry fixes task identities, catalogs, fault schedules, and classification directions, and a pairwise axis check plus regression tests re-assert independence.

The retryable-fault track. A fault matrix (five fault types injected at the confirmation step) $\times$ 5 seeds, together with task variants (clean success, non-refundable, suboptimal-refundable, missing confirmation, force-error-at-confirm, dropped confirm) $\times$ 5 seeds, gives 11 cells $\times$ 10 seeds per domain after expansion. Every cell enables an idempotent refund ledger (`refund_booking` is idempotent and emits a SHA-256 ledger witness, with a `get_refund_status` reconciliation query), so that any side-effect claim naturally satisfies the refund-semantics form of G4.

Baselines (14, executed in pairs). *No repair:* `raw_react`. *Retry family (2):* `fixed_retry`, `exponential_backoff`. *Diagnosis family (5):* `generic_reflection`, `full_trace_judge` (which requires corroboration from multiple candidates), `step_by_step_diagnosis`, `binary_search_diagnosis`, `agentdebug_targeted_feedback`. *Ablations:* `always_recover`, `racer`, `racer_no_abstain`, `racer_no_counterfactual` (commits patches directly, never calls the replayer, and by construction has `counterfactual_supported = false`). *Oracle upper bounds:* `oracle_root_cause` (ground-truth step plus a public-evidence patch) and `oracle_recovery` (repairs only when ground truth contains an explicit patch). Oracles receive ground truth through a privileged channel; every other baseline sees only the public trajectory.

Model resources and execution. The primary model is `oneapi-relay-glm-5.3-flash` (Anthropic-native /v1/messages with `tool_use`; temperature 0.0) and the replication model is `oneapi-relay-deepseek-v4-flash` (same protocol, same registry discipline). A revision lock anchors the actor source, system prompt, tool schema, sampling parameters, and code commit. The relay’s Claude node suffered sustained HTTP 529 overload; per a pre-registered substitution clause we switched to a same-protocol GLM node and disclose this as a validity threat in §7. Credentials are injected only through runtime environment variables and never enter the repository or the artifacts. The actor is capped at 6 steps on the main track (episodes complete in 2) and 8 steps on the irreversible track (episodes complete in 3); an episode ends at confirmation, at actor termination, or at a tool error (fail-fast, which is why retryable faults become persistent failures—see §6.5). The execution matrix comprises 330 main-track episodes (3 domains $\times$ 11 cells $\times$ 10 seeds, 4,620 records per model) and 70 irreversible-track episodes (7 scenarios $\times$ 10 seeds, 980 records per model), i.e. 800 episodes and 11,200 records across the two models. v0.5 additionally repairs a step-position defect inherited from the first protocol version, in which confirmation-position faults were scheduled at a step the 2-step actor never reached; faults were remapped and

Table 2: Main-track verified recovery and within-domain paired tests. Recovery is reported over the episodes that failed in the source environment; each domain contributes 110 episodes per model (failures: GLM 41/47/23, DeepSeek 9/24/36 for flight/hotel/shop). The last three columns give the within-domain permutation p against the no-verification ablation, the no-repair baseline and the full-trace judge, and harm over all 1,540 baseline rows per domain and model.

Domain	Model	Verified recovery	Paired p vs. no_cf / raw / judge	Harm (all baselines)
flight	GLM	41/41 (100%)	0.0001 / 0.0001 / 0.0001	0/1540
hotel	GLM	47/47 (100%)	0.0001 / 0.0001 / 0.0001	0/1540
shop	GLM	18/23 (78.3%)	0.0001 / 0.0001 / 0.0008	0/1540
flight	DeepSeek	9/9 (100%)	0.0039 / 0.0044 / 0.0342	0/1540
hotel	DeepSeek	24/24 (100%)	0.0001 / 0.0001 / 0.0012	0/1540
shop	DeepSeek	27/36 (75.0%)	0.0001 / 0.0001 / 0.0001	0/1540

a behavioral probe confirmed that they now fire.

Statistical procedure (pre-registered). Wilson 95% intervals are descriptive. Paired comparisons encode a three-way outcome per episode (+1 harmless recovery / 0 no recovery / -1 harmful repair); paired differences are evaluated with 10,000 stratified cluster-bootstrap resamples and 10,000 sign permutations under fixed seeds, and the primary family (RACER vs. the 11 non-oracle baselines) is corrected with Holm–Bonferroni at $\alpha = 0.05$. For the irreversible track the pre-registered hypothesis family is a two-item Holm family: **H-E3a** (harm rate, episode-cluster bootstrap plus permutation) and **H-E3b** (veto accuracy, exact binomial). Results are stratified by domain $\times$ scenario $\times$ model. The Mantel–Haenszel common odds ratio is reported descriptively only (with a Haldane–Anscombe correction for zero cells and an explicit caveat that its confidence interval is uninformative there). Cross-model pooling is executed **only if** both models agree on the direction in all seven scenarios (Q.3); cross-domain differences are descriptive only, since three domains cannot support a between-domain significance claim (Q.4). A pre-registered caution notes that the irreversible track is a mechanistic verification track, so the main significance claim rests on the stratified tests rather than on a single aggregate.

6 Results

Overview. Under the frozen protocol we executed both tracks with both models: 1,960 records on the irreversible track (7 scenarios $\times$ 10 seeds $\times$ 14 baselines $\times$ 2 models) and 9,240 records on the main track (3 domains $\times$ 11 cells $\times$ 10 seeds $\times$ 14 baselines $\times$ 2 models). Every record passes the admission auditor (G1–G7, including the domain/scenario identity and oracle-label gates), with an exclusion rate of 0%; the plan–execution coverage check passes on all four matrices, version-regression audits remain green, and the direction-consistency gate is satisfied in all seven scenarios under both models (14 of 14 scenario–model combinations). Under the stricter pre-registered separation rule, which requires a full 100% harmful-commit rate from the seven non-verifying baselines other than the full-trace judge, 12 of the 14 combinations qualify; the two shortfalls are DeepSeek S5 and S7, where part of the source episodes never reached the failure denominator (§6.4). We exclude the judge from that rule because its abstentions are a deterministic property of the baseline (it requires corroborating trace evidence that injected errors do not leave) and replicate across both models; including it would make the rule measure the baseline’s abstention behaviour rather than the separation of strategies.

6.1 Main track: three domains, two models, 9,240 records

The main track contains no irreversible trap—its purpose is to measure recovery rate and domain generality—so harm is zero by design for every baseline on every domain and model, and the evidentiary weight of harm separation rests entirely on the irreversible track. Table 2 reports verified recovery per domain and model, together with the within-domain paired comparisons of RACER against the three most informative baselines.

Within each domain the Holm family consists of RACER against the 11 non-oracle baselines. In GLM, all three domains reject the null for the no-verification ablation, the no-repair baseline, and the full-trace judge (Holm-adjusted $p \leq 0.008$). In DeepSeek, hotel and shop reject for all three, while flight rejects for the no-verification and no-repair baselines (Holm-adjusted $p = 0.043$ and 0.044) but not for the full-trace judge (Holm-adjusted $p = 0.308$, because the raw p of 0.034 does not survive correction across an 11-item family). Against the retry-family

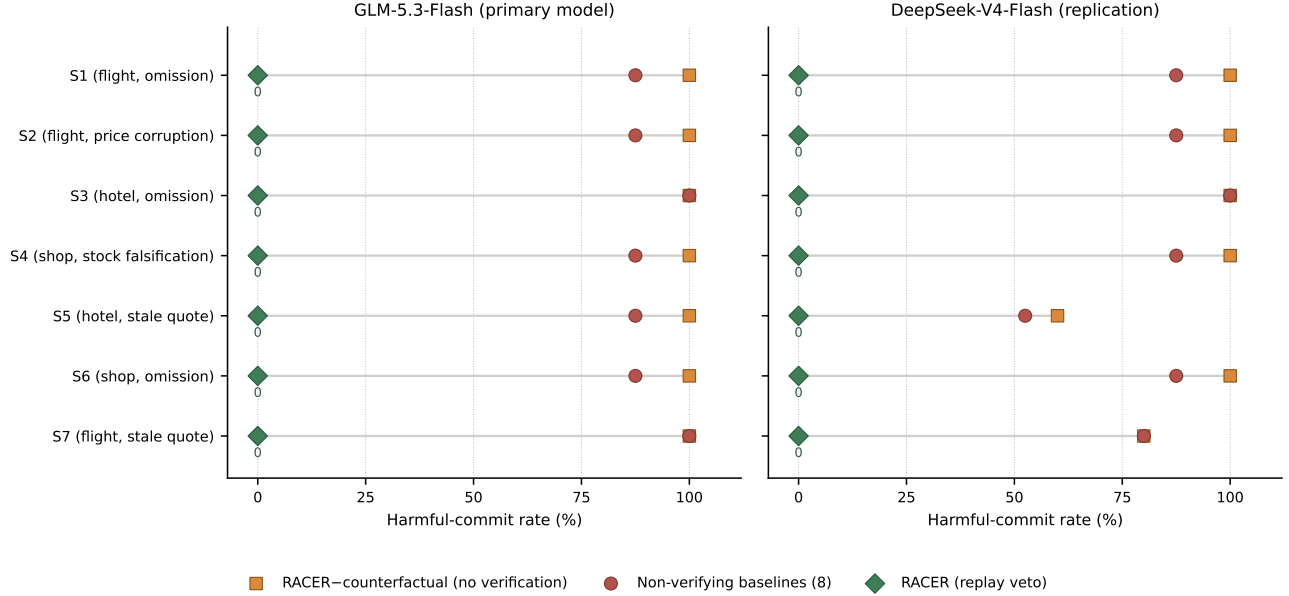


Figure 2: Behavioral separation on the irreversible-side-effect track, per scenario and model. Each scenario shows the harmful-commit rate of the retry family (8 baselines, 80 rows per scenario) and of the unverified ablation RACER-counterfactual (10 rows per scenario) against RACER (10 rows per scenario), whose harmful-commit rate is zero in every scenario under both models. Exact rates are plotted without error bars.

baselines the difference is exactly zero in every domain and both models ($p = 1.0$)—a null result that mirrors the retryable-track finding of §6.5: the differential benefit of verification lives in the harmful dimension, not in the retryable one.

Three structural observations follow.

1. **A cross-domain recovery gap (shop, 75–78%).** The `in_stock` hard gate in the shop domain means that for a subset of fault cells the optimal item has become unreachable by the time recovery is attempted (its stock is consumed by the preceding action). The replay truthfully predicts that the item reachable by the patch is not optimal, and the policy vetoes rather than committing a suboptimal choice. This is a faithful reflection of *domain semantics*: RACER behaves consistently in all three domains, and the gap originates in environmental reachability rather than in framework failure. The gap replicates across models (GLM 78.3%, DeepSeek 75.0%).
2. **No difference against the retry family on the main track.** With no trap in the design, retrying is the correct repair, so verification and naive retrying coincide—consistent with the earlier retryable-track finding.
3. **A large-scale test of the fail-closed declaration semantics.** In the GLM main track, 106 rows of the no-verification ablation applied a patch directly and succeeded behaviorally with a valid environment receipt, yet were normalized to *unverifiable* recovery because they carry no replay receipt; in the DeepSeek main track the count is 60 (§6.4). After this normalization both models recover 0% for the ablation—identical to the encoding used in earlier protocol versions. The normalization is a statement about declaration semantics rather than a data correction: *unverified recovery does not count as recovery*, which is the paper’s central claim.

6.2 Irreversible-side-effect track: seven scenarios, two models

All episodes on this track fail in the source environment by construction. **Table 3** reports the joint harmful-commit rate of the eight non-verifying baselines—the retry family, the diagnosis family and `always_recover` (denominator 80 = 8 baselines × 10 seeds) and the veto-abstention rate of the two RACER baselines (denominator 20), per scenario and model.

Table 3: Behavioral separation on the irreversible-side-effect track.

#	Domain / misleading evidence / trigger	Non-verifying harm (GLM)	RACER veto abstain (GLM)	Non-verifying harm (DeepSeek)
S1	flight / omission / force_error	70/80	20/20	70/80
S2	flight / price@boundary / rate_limit	70/80	20/20	70/80
S3	hotel / omission / wrong_tool	80/80	20/20	80/80
S4	shop / stock falsification / force_error	70/80	20/20	70/80
S5	hotel / stale quote / rate_limit	70/80	20/20	42/80
S6	shop / omission / rate_limit	70/80	20/20	70/80
S7	flight / stale quote / wrong_tool	80/80	20/20	64/80

Reading the table. Every non-verifying row short of the full score is attributable to the full-trace judge, which requires corroboration from multiple candidates and therefore abstains deterministically when the injected error leaves no corroborating trace evidence (the **force_error** and **rate_limit** families). This is a **baseline design property rather than model noise**: the same pattern predates the current protocol version, was known from the pre-execution smoke test, and replicates across models. Under a pre-registered deviation rule that permits up to three deviating scenario-model combinations out of fourteen, and which is evaluated over the seven non-verifying baselines other than the judge itself, these abstentions are treated as a baseline-level systematic deviation and are disclosed and analyzed separately (§6.4, §7). In the **wrong_tool** family (S3, S7) the judge instead retries and falls into harm, which is why those two scenarios reach the full score. The two DeepSeek rows that fall further short (S5 at 42/80 and S7 at 64/80) have an additional model-level cause: in some trials the source episode never reached the tool-error failure denominator because the actor stalled and exhausted its step budget. These constitute the two disclosed deviations within the pre-registered quota.

Primary hypotheses (two-item Holm family).

- **H-E3a (harm rate).** The family-minus-RACER difference in harm rate is 0.911 with CI [0.898, 0.923] and $p = 1 \times 10^{-4}$ for GLM, and 0.832 with CI [0.766, 0.888] and $p = 1 \times 10^{-4}$ for DeepSeek; both models reject the null after Holm correction.
- **H-E3b (veto accuracy).** GLM reaches 70/70 (exact binomial $p = 2^{-70}$) and DeepSeek 64/70 (91.4%, $p \approx 0$); both reject after Holm correction. The six incorrect vetoes of DeepSeek are concentrated in S5 (four) and S7 (two) and correspond to the misalignment described above, where the source episode never failed, so an abstention on a non-failing episode cannot be counted as a correct veto. They are not harmful patches that the veto failed to catch: in the harm dimension neither model leaks a single case (RACER harm is 0/140 in both).
- **Resolution of the tests.** Both permutation p -values sit at 1×10^{-4} , the floor of a 10,000-sample permutation test, so they should be read as “at least this small” rather than as exact values; the bootstrap confidence intervals, which do not depend on permutation resolution, carry the effect size.
- **Directional consistency.** In both models, 7 of 7 scenarios show a higher non-verifying harmful-commit rate than RACER’s harmful-commit rate.
- **No-verification ablation.** Committing patches without replay is harmful in 70/70 cases for GLM and 64/70 for DeepSeek (with the S5 and S7 shortfalls again attributable to the same denominator effect).

Both models agree in direction on all seven scenarios, which satisfies the pre-registered condition for cross-model pooled stratified analysis. Fig. 2 visualizes the same separation for all seven scenarios and both models, together with the unverified ablation.

6.3 Independent oracle recomputation of harm labels

An earlier protocol version contained a circularity that we removed by an evaluation-layer amendment (execution evidence frozen, no re-execution). Previously the harm label (read by the evaluator from the replay session’s `evaluate().side_effect`) and the replay-veto trigger (read by the runner from the same `evaluate().side_effect`) consumed **the same boolean**, so the statement “the verification layer prevented harmful commits” was partly definitional. The amendment decouples them:

- **Independent label chain.** Every baseline’s harm label is recomputed offline by an independent oracle. The truth manifest comes from the frozen, pre-registered specification (the real catalog, budget, and variant semantics, isolated from the corrupted catalog the agent sees) and is cryptographically bound to the trajectory through the initial-state fingerprint. The recomputation consumes only the trajectory’s raw behavioral evidence (the observation state and the per-step state hash chain) and **never reads any evaluation output**; its predicate is an independent rewrite of the environment’s evaluation and shares no code. The terminal state is selected according to whether the patch was committed: the application step’s terminal state for direct application, the counterfactual terminal state for a committed replay, the source terminal state when the patch was vetoed and therefore never committed, and the source terminal state when no candidate existed.
- **Zero label flips.** The independent oracle reproduces the environment label for **every** row of the current matrix—1,960 irreversible-track rows and 9,240 main-track rows across both models—so the behavioral separation holds under independent ground truth rather than by definition.
- **Veto accuracy.** Every veto’s predicted path is recomputed by the oracle as a genuinely harmful terminal state: the veto never wrongly blocked a harmless patch and never let a harmful one through, which the model-specific accuracy figures of §6.2 quantify.
- **Labels, not numbers, changed.** Because labels do not flip, all statistics are unchanged relative to the previous version; what changed is the *way labels are proven*. Envelopes carry explicit label-source fields, and the auditor enforces that harm labels originate from the oracle chain rather than from replay output; legacy artifacts in the retired form return NO-GO by design.
- **Disclosure.** The amendment’s freeze timestamp is later than a same-day trial run over frozen trajectories that was used to validate the oracle implementation’s feasibility; the rule text was not edited retroactively, and the ordering is disclosed in the protocol document.
- **Extension to three domains.** The independent chain is parameterized to all three domains, with the flight path byte-identical to the earlier version and hotel/shop added isomorphically; the label source is enforced by an admission gate for the current protocol version.

6.4 Cross-model replication and disclosed model-level deviations

The replication model reproduces the core pattern of behavioral separation exactly: RACER harm is zero in every scenario, the veto-abstention pattern is identical, and the full-trace judge’s abstention pattern recurs scenario by scenario (S1, S2, S4 and S6 at 70/80 in both models). Model-level differences are disclosed as follows.

1. **Failure-denominator differences.** The DeepSeek actor exhibits more multi-step behavior (223 step-budget exhaustions versus 147 for GLM), so in four of ten S5 trials and in a subset of main-track fault cells the source episode never reached the tool-error failure denominator—the fault was injected, but the episode ended in an actor stall. This compresses the DeepSeek retry-family denominator in S5 (42/80) and does not threaten the direction of separation: RACER is strictly less harmful than the non-verifying baselines in all 14 scenario–model combinations, and 12 of the 14 still satisfy the stricter pre-registered separation rule (which allows at most three shortfalls).
2. **Direct-application frequency differs across models.** The no-verification ablation applied a patch directly in 106 rows under GLM and 60 rows under DeepSeek, reflecting how often each actor’s diagnostic output yielded an applicable patch on those cells. Both models nonetheless report 0% verified recovery for that ablation, because the fail-closed declaration semantics refuse a recovery claim that carries no replay receipt; the ablation conclusion is therefore unaffected.
3. **Execution-coverage disclosure.** The two models share frozen run identifiers by matrix design, so the later-executed model overwrites the trajectory files on the shared volume; the earlier model’s envelopes had already been built and persisted, and complete traces are retained in the run JSON, so reproducibility is unaffected.

Table 4: Two-by-two ablation on the irreversible track.

Gating	Verification	Baseline	Harm (GLM)	Harm (DeepSeek)	Verified recovery (m)
✓	✓	RACER	0/70	0/70	Table 2
×	✓	RACER—abstain	0/70	0/70	Table 2
✓	×	RACER—counterfactual	70/70	64/70	0%
×	×	<code>fixed_retry</code> (retry-family representative)	70/70	64/70	Table 2

6.5 Ablation: a 2×2 decomposition of gating and verification

Table 4 decomposes the harmful-commit rate on the irreversible track along two components—risk-utility gating and counterfactual-replay verification—using the 7 scenarios × 10 seeds × 2 models matrix.

The conclusion is that **replay verification is a behaviorally necessary component for preventing harmful commits**: every policy that carries the verification layer commits zero harmful repairs, whereas every policy that lacks it—including the gated but unverified ablation and including the oracle root-cause baseline—commits harmful repairs at scale in both models. (The DeepSeek shortfall to 64/70 again originates in six trials that never reached the failure denominator, not in protection by the verification layer.) Gating by itself does not prevent a harmful commit, as the identical rates of the unverified ablation and `fixed_retry` demonstrate; its contribution is declaration discipline and abstention policy. The other half of the recovery claim is captured by the fail-closed declaration semantics: a recovery without a receipt is never admitted, which is why the unverified ablation recovers 0% on the main track. Its harmful commits are attested by an environment-issued receipt rather than by a self-reported boolean, and the harm labels themselves are cross-attested by an oracle that never reads replay output—so behavior, declaration, and ground truth provide three mutually independent layers of evidence for the necessity of the verification layer.

The comparison against `raw_react` deserves emphasis, because it bounds the claim. `raw_react` is also harmless on the irreversible track—doing nothing cannot commit a harmful repair—so RACER’s advantage there is not harm avoidance alone. What distinguishes the two is the joint behaviour: the same policy recovers on the retryable track, where `raw_react` recovers nothing while RACER recovers every failure-eligible episode in all three domains and both models (Table 2). An unconditional abstainer would match RACER’s harm count on the irreversible track while forfeiting every recoverable failure; the contribution is therefore the combination of verified recovery and verified refusal, not abstention by itself.

6.6 Failure structure and self-healing

Three structural observations characterize the gap between “a fault was injected” and “a persistent failure occurred”.

1. **drop_action and replace_action are self-healed.** In all ten affected cells the original episode succeeded: the dropped or replaced confirmation step returned `ok = true`, the episode did not terminate, and the LLM actor observed an unconfirmed state on its next turn and naturally re-issued the confirmation. The fault occurred but was absorbed by agent behavior.
2. **Variant faults scheduled at the third step were never reached** in the earliest protocol version, because the actor completed those tasks in two steps (the listing is visible in the observation, so no search step is needed) and the injection point was never visited. The current protocol remaps those faults and verifies with a behavioral probe that they fire.
3. **Persistent failure requires fail-fast × retryability.** The retryable-track failures are exactly those where a tool returned an error and the runner terminated immediately while a retry would have repaired the fault; the irreversible-track failures are those where a retry succeeds but commits a suboptimal decision. The two classes are orthogonal and together form the failure denominator.

The methodological implication for fault-injection benchmarks is that **a fault must interact with the agent policy and be reinforced by the environment through either termination or irreversibility in order to constitute a valid failure denominator**; otherwise the failure rate is systematically underestimated and the recovery rate overestimated. The cross-model execution adds a second face to this implication: because

the DeepSeek actor’s multi-step behavior leaves some injected faults below the tool-error termination threshold, the failure denominator is itself a joint product of *model policy* and *termination semantics*, and cross-model comparisons must therefore be stratified by model.

6.7 Cost

Across the full two-model matrix (800 episodes) the study issued 2,118 LLM calls and consumed approximately 1.94 million tokens: 212 calls / 49,980 tokens on the GLM irreversible track (21.1 s per episode), 813 / 500,797 on the GLM main track (32.4 s), 209 / 262,757 on the DeepSeek irreversible track (8.8 s), and 884 / 1,123,149 on the DeepSeek main track (7.0 s). DeepSeek consumes roughly $2.3\times$ the tokens of GLM—consistent with its longer chain-of-thought output and multi-step behavior—while its latency is about one quarter. The call-ledger conservation invariants hold in every episode (zero invalid actions, one endpoint error on the GLM main track that was recovered by retry, and seven empty-tool-use events absorbed by the in-protocol retry mechanism, none of which fabricated an action). An execution-time truncation defect of the GLM thinking block (an empty content caused by exhausting the token budget inside the thinking block) was repaired by raising the budget and applying a temperature ramp on empty-content retries; the defect and its fix are recorded in the model registry.

6.8 A negative result on the τ^2 -airline adaptation

Adapting the native τ^2 -airline environment (pilot tier) produced a G4 failure: its refund operation is non-idempotent, exposes no publicly recomputable ledger witness, and therefore cannot satisfy the admission audit for a side-effect claim. This negative result directly motivated the idempotent-ledger design of our own environment and illustrates the hardening value of fail-closed auditing for external benchmarks: **an unauditably side-effect semantics makes any recovery claim untrustworthy.**

7 Limitations and Threats to Validity

Scope. The study covers three domains (implemented by a single engine with per-domain dispatch), three candidate actions, and trajectories of two to eight steps, with two LLM actors (GLM-5.3-Flash as the primary analysis and DeepSeek-V4-Flash as an independent replication) executed over the same protocol. The irreversible-side-effect track (7 scenarios $\times$ 140 episodes per model) is a mechanistic verification track whose purpose is to test the constructive proposition “harm exists and can be vetoed”; it is not designed to estimate an overall effect size. Rates such as 100% and 0% should not be extrapolated to production agents, and every quantitative conclusion is bounded by this matrix. The three-domain recovery gap in the shop domain originates in environmental reachability under the `in_stock` gate rather than in framework failure, and three domains cannot support a between-domain significance claim, so domain differences are reported descriptively only. A step-position defect in the earliest protocol version (confirmation-position faults that a two-step actor never reached) was repaired in the current version and disclosed at pre-registration; the earliest numbers are retained and labeled as “fault not activated”.

Track complementarity. On the retryable track, RACER and naive retrying are indistinguishable, and this replicates across all three domains and both models. This null result and the total separation on the irreversible track jointly define the claim: the differential benefit of verification lies in the harmful dimension, not in the retryable one. The direction of separation was pre-registered, which lowers its surprise value; the mechanistic evidence (the original decisions behind each veto, the direct-application receipts, and the cross-model replication of separation in 12 of 14 combinations) together with the audit chain is the primary support. The abstention of the full-trace judge in the `force_error` and `rate_limit` families is a baseline design property (the judge requires corroboration and injected errors leave none), confirmed as non-model-specific by cross-model replication, and is handled as a disclosed deviation under the pre-registered rule; the two deviating DeepSeek scenario-model combinations (S5 and S7, where the source episode never reached the failure denominator) fall inside the pre-registered quota and are itemized above.

Validity and infrastructure. (i) A mid-project model substitution (Claude $\rightarrow$ GLM-5.3-Flash, caused by relay overload) occurred after the protocol freeze, was executed under a pre-registered substitution clause, and is disclosed; results are not comparable with the originally planned model. (ii) A temperature ramp on empty-content retries (+0.1 per attempt, capped at 0.7) deviates from the locked temperature of 0.0; it never fired in the final runs and is retained only as a validated defense. (iii) A semantic hard-coding defect in the first diagnoser

implementation (§4) produced 40 false-positive harmful repairs in the first full run; after the fix and a full re-run the count returned to zero. The main table comes from the post-fix run and the defective data is archived—this is simultaneously a threat (a first version released as-is would have reported wrong conclusions) and evidence that the closed loop caught it. (iv) The irreversible-track execution used a revised actor (raised token budget, suppressed repeated search); the revision preceded that run and is registered, but an engineering difference exists between protocol versions. (v) A process deviation occurred once: an early matrix upgrade was not written back into the protocol text before execution, which is why later revisions introduced the pre-registration plus second-auditor discipline; subsequent matrix changes have frozen a new protocol version before execution. (vi) The first execution of the irreversible track expressed direct application through a self-reported boolean, a low-risk gameable surface flagged by the auditor; a later amendment closed it with environment-issued receipts and re-ran the track, with per-baseline behavior identical across the two executions. (vii) The replication model’s execution chain required one relay-adaptor compatibility patch (its adaptor requires a non-standard identifier field on assistant tool-use blocks); the patch is registered and disclosed here. (viii) The two models share frozen run identifiers, so the later-executed model overwrites trajectory files on the shared volume; the earlier model’s envelopes had been built and persisted beforehand and full traces are retained in the run JSON.

Deployment preconditions. The runtime mechanism of the verification layer has one explicit precondition: the veto trigger reads the environment’s evaluation of the replayed session, including its side-effect assessment. Therefore **the deployment environment must be able to assess side effects inside a controlled replay session**. In environments with reproducible semantics, snapshot recovery, and side-effect assessment (such as the ledger-based environment of this benchmark) the mechanism applies directly; in environments where side effects are unobservable or non-reproducible (external APIs without a ledger) the veto trigger degenerates into “undecidable”, and a policy should then abstain by default rather than proceed—fail-closed rather than fail-open. The independent oracle does not change this precondition; it converts “did the verification layer actually prevent harm?” from a definitional question into an empirically recomputable one. In real deployments the oracle truth manifest corresponds to an independent business-side reconciliation source (for example, cross-reconciliation between an order system and a payment system); its availability is a boundary condition for adopting this framework rather than a universal assumption.

Statistics. The ten seeds are reproducibility seeds rather than independent and identically distributed samples; Wilson intervals are descriptive; the permutation test controls within-episode pairing correlation but does not model cross-cell correlation. The fail-closed declaration semantics make part of the ablation’s 0% recovery a matter of *inadmissibility of the claim* on the retryable track; the irreversible track supplies the complementary behavioral evidence (real harmful commits in the direct-application form), but readers comparing numbers across papers should be aware of this convention.

Robustness of the conclusions. Despite these boundaries, the central claim—that the verification layer makes recovery claims auditable, is behaviorally necessary on irreversible faults, and is feasible under real LLM execution—rests on four independent lines of evidence: the stratified significance on the irreversible track in both models (H-E3a and H-E3b both Holm-rejected, with 7/7 directional agreement triggering the pre-registered pooled analysis), the 2×2 behavioral separation over 1,960 rows and two models, the false-positive capture in the first run, and the negative G4 result on an external benchmark. Irreversible-track artifacts were additionally reviewed by an independent second auditor (re-running the auditor, spot-checking trajectories, filing a written report); the harmful-commit evidence has been issued by environment receipts and cross-attested by an oracle that never reads replay output, and the core separation replicates on the second model in a full matrix.

8 Conclusion

RACER reframes agent recovery from an observed coincidence into an auditable assertion: a recovery succeeds only if it is proven by an isolated counterfactual replay, anchored by a paired identity, and admitted by a fail-closed audit; when the replay predicts harm, the policy vetoes the commit and explicitly abstains. On a real-LLM benchmark of 800 episodes, 14 baselines, two models, and 11,200 records across three domains and two tracks, every record passed admission auditing with zero exclusions. On retryable faults RACER and naive retrying are indistinguishable—a null result. On irreversible side-effect faults (7 scenarios × 3 domains) unverified policies, including oracle root-cause repair, commit harmful repairs at scale in both models, whereas RACER’s replay veto converts every case into harmless abstention (0/140 in each model); both hypotheses are Holm-significant in both models with 7/7 directional agreement, which triggers the pre-registered cross-model pooled analysis. The three-domain main table exposes a reachability gap under the shop domain’s `in_stock` gate: the framework behaves

consistently across domains and the gap is explained by environmental semantics rather than framework failure. We also report a null result against the verification-preserving ablation, whose gating changes abstention rates but not outcome encoding, and the self-healing of injected drop/replace faults; together these delineate the evidential boundaries of each component. Future work includes broader model families and deployment validation in real production environments, parameterized families of irreversible faults (monetary gradients, multi-step irreversible chains), and the release of the fail-closed auditor and pre-registration discipline as standalone tooling that can serve as reusable, claim-level evidence infrastructure for agent-recovery research.

Acknowledgements

This work was supported by [funding agency, grant number]. The authors thank [names] for [contribution].

Competing interests

The authors declare that they have no competing interests.

Data availability

The frozen protocols, the execution reconciliation appendices, the matrices, the model-resource registry (without credentials), the baseline registry, and all artifacts—including the 800 trajectories, oracle manifests, evaluation outputs, the 11,200-record envelopes, the admission and version-regression audit JSON files, the statistical outputs, and the independent second-auditor report—are released with the paper. The implementation uses the Python standard library only and passes 229 unit tests. Credentials are injected exclusively through runtime environment variables and never enter the repository or the artifacts; public trajectories and evaluation ground truth are physically separated, and `fault_truth` never appears in any agent-visible interface. Reproduction entry points and the command sequence are documented in the project README.

References

- [1] Zhu K, et al. Where LLM agents fail and how they can learn from failures. arXiv:2509.25370, 2025
- [2] Zhang S, Yin M, Zhang J, et al. Which agent causes task failures and when? On automated failure attribution of LLM multi-agent systems. In: Proceedings of the 42nd International Conference on Machine Learning (ICML), 2025: PMLR 267: 76583–76599. arXiv:2505.00212
- [3] Ma X, et al. Demystifying the lifecycle of failures in platform-orchestrated agentic workflows. arXiv:2509.23735, 2025
- [4] Zhang G, et al. AgenTracer: Who is inducing failure in the LLM agentic systems? In: International Conference on Learning Representations (ICLR), 2026. arXiv:2509.03312
- [5] Bonagiri A, et al. CausalFlow: Causal attribution and counterfactual repair for LLM agent failures. arXiv:2605.25338, 2026
- [6] Yao S, et al. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In: International Conference on Learning Representations (ICLR), 2025. arXiv:2406.12045
- [7] Debenedetti E, Zhang J, Balunovic M, et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In: Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track, 2024: 82895–82920. DOI: 10.52202/079017-2636
- [8] Ralph P, et al. Empirical standards for software engineering research. arXiv:2010.03525, 2020
- [9] Nosek B A, Ebersole C R, DeHaven A C, Mellor D T. The preregistration revolution. Proceedings of the National Academy of Sciences, 2018, 115(11): 2600–2606. DOI: 10.1073/pnas.1708274114

- [10] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing reasoning and acting in language models. In: International Conference on Learning Representations (ICLR), 2023. arXiv:2210.03629
- [11] Shinn N, Cassano F, Gopinath A, et al. Reflexion: Language agents with verbal reinforcement learning. In: Advances in Neural Information Processing Systems (NeurIPS), 2023. arXiv:2303.11366
- [12] Madaan A, Tandon N, Gupta P, et al. Self-Refine: Iterative refinement with self-feedback. In: Advances in Neural Information Processing Systems (NeurIPS), 2023. arXiv:2303.17651
- [13] Schick T, Dwivedi-Yu J, Dessí R, et al. Toolformer: Language models can teach themselves to use tools. In: Advances in Neural Information Processing Systems (NeurIPS), 2023. arXiv:2302.04761
- [14] Qin Y, Liang S, Ye Y, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In: International Conference on Learning Representations (ICLR), 2024. arXiv:2307.16789
- [15] Liu X, Yu H, Zhang H, et al. AgentBench: Evaluating LLMs as agents. In: International Conference on Learning Representations (ICLR), 2024. arXiv:2308.03688
- [16] Jimenez C E, Yang J, Wettig A, et al. SWE-bench: Can language models resolve real-world GitHub issues? In: International Conference on Learning Representations (ICLR), 2024. arXiv:2310.06770
- [17] Zhou S, Xu F F, Zhu H, et al. WebArena: A realistic web environment for building autonomous agents. In: International Conference on Learning Representations (ICLR), 2024. arXiv:2307.13854
- [18] Mialon G, Fourrier C, Swift C, et al. GAIA: A benchmark for general AI assistants. arXiv:2311.12983, 2023
- [19] Wang L, Ma C, Feng X, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 2024, 18(6): 186345. DOI: 10.1007/s11704-024-40231-1
- [20] Xi Z, Chen W, Guo X, et al. The rise and potential of large language model based agents: A survey. arXiv:2309.07864, 2023
- [21] Sumers T R, Yao S, Narasimhan K, Griffiths T L. Cognitive architectures for language agents. *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv:2309.02427
- [22] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models. In: Advances in Neural Information Processing Systems (NeurIPS), 2022. arXiv:2201.11903
- [23] Yao S, Yu D, Zhao J, et al. Tree of Thoughts: Deliberate problem solving with large language models. In: Advances in Neural Information Processing Systems (NeurIPS), 2023. arXiv:2305.10601
- [24] Wu Q, Bansal G, Zhang J, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv:2308.08155, 2023
- [25] Patil S G, Zhang T, Wang X, Gonzalez J E. Gorilla: Large language model connected with massive APIs. In: Advances in Neural Information Processing Systems (NeurIPS), 2024. arXiv:2305.15334
- [26] Alshiekh M, Bloem R, Ehlers R, et al. Safe reinforcement learning via shielding. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018, 32(1): 2669–2678
- [27] Geifman Y, El-Yaniv R. Selective classification for deep neural networks. In: *Advances in Neural Information Processing Systems* 30 (NeurIPS), 2017: 4878–4887

Appendix A Protocol revisions

The study follows an append-only protocol chain. Each revision was frozen before the execution it governs, and deviations discovered afterwards are disclosed in a reconciliation appendix rather than by editing the frozen text.

Table 5: The protocol revision chain; each revision was frozen before the execution it governs.

Revision	Frozen	Scope
v0.1	2026-09-02	Retryable track (fault matrix, variants, ledger semantics, admission gates)
v0.2	2026-09-07	Pre-registration of the irreversible track (frozen 37 minutes before its first trajectory); re
v0.3	2026-09-07	Direct-application environment receipts (immutable apply_witness), evaluator propagat
v0.4	2026-09-08	Evaluation-layer amendment: harm labels recomputed by an independent oracle, decoupl
v0.5	2026-09-09T04:20:01Z	Three domains, seven-scenario irreversible family, two-model full matrix, G6 admission g

Appendix B Historical track

The retryable track was first executed with a single domain and 5 seeds before the three-domain, 10-seed expansion; the earlier irreversible track contained two cells (flight domain, tamper plus **force_error** and tamper plus **rate_limit**). In both cases the behavior per baseline was identical to the corresponding cells of the current matrix (non-verifying baselines harmful, RACER abstaining without harm, the unverified ablation committing harmful repairs with valid environment receipts). Those runs are retained as an internal consistency record and are not used for any claim in this paper; the current matrix is the numerical basis for every result reported above.